Affect Recognition using Multi-Task Learning

This project implements a deep learning pipeline for facial affect recognition. It trains models to perform two tasks simultaneously:

1. **Expression Classification:** Categorizing facial expressions (e.g., happy, sad, angry).
2. **Valence-Arousal Regression:** Predicting continuous values for emotional valence (pleasure/displeasure) and arousal (intensity of emotion).

The pipeline uses pre-trained ResNet50 and VGG16 architectures as backbone encoders, fine-tuning them for these specific multi-task objectives. The code is designed to be run in a Google Colab environment.

Setup and Data Extraction

This section handles the initial environment setup and data preparation.

1. Imports: The script begins by importing necessary libraries for file operations (os, zipfile, glob, pathlib), numerical operations (numpy), data handling (json), deep learning with PyTorch (torch, torch.nn, torch.optim, torch.utils.data, torchvision.transforms, torchvision.models, torch.nn.functional), image processing (PIL), random operations (random), and metric calculations from scikit-learn (sklearn.metrics) and scipy (scipy.stats).

2. Data Extraction: It expects a .zip file named DL_Assignment1_Dataset.zip (or DL_Assignment1_Dataset (1).zip) to be uploaded to the /content/ directory in Google Colab. This zip file is then extracted into /content/data. This directory is expected to contain an images folder and an annotations.npy file.

3. Configuration: A CONFIG dictionary is defined to manage key parameters for the experiment. This includes:

- DATA_DIR: Path to the extracted image directory.
- ANNOT_NPY: Path to the NumPy array containing annotations.
- Hyperparameters: BATCH_SIZE, NUM_EPOCHS, LR (learning rate), WEIGHT_DECAY.
- System settings: NUM_WORKERS (for data loading), DEVICE (automatically set to cuda if available, else cpu), OUTPUT_DIR (for saving results).
- Dataset split: VAL_SPLIT (percentage for validation), RANDOM_SEED.
- Loss weights: CLASS_WEIGHT, REG_WEIGHT (for balancing the two tasks' losses).

4. Output Directory Creation: The script ensures that the OUTPUT_DIR specified in CONFIG exists.

5. Data Verification (Colab Specific): Several blocks of code are included to verify the data extraction and file paths, which is particularly useful in dynamic environments like Colab where file locations can sometimes be tricky. It lists contents of common directories, searches for .npy, .npz, and .zip files, and attempts to load the annotations.npy file to confirm its integrity. It also includes an auto-detection mechanism for the annotations file if the specified path doesn't exist.

Dataset Exploration and Custom Dataset

This section delves into understanding the dataset structure and preparing it for PyTorch.

1. Annotation Loading: The annotations.npy file is loaded using numpy.load(..., allow_pickle=True). This file likely contains mappings or metadata about the image samples and their associated labels.

2. Annotation Directory Structure: The script then explicitly looks for annotations within /content/data/Dataset/Dataset/annotations. It identifies four types of annotation files:

- *_exp.npy: Expression labels (e.g., 0-7 for different emotion categories).
- *_val.npy: Valence labels (continuous values).
- *_aro.npy: Arousal labels (continuous values).
- *_lnd.npy: Facial landmark coordinates (NumPy arrays).

It groups these files by type and creates dictionaries mapping sample IDs to their respective annotation file paths. Finally, it determines sample_ids which are present across all four annotation types, ensuring complete data for each sample.

3. AffectDataset Class: A custom PyTorch Dataset class, AffectDataset, is defined to handle loading images and their corresponding annotations.

- **__init__**:
    - Takes img_dir, ann_dir, and an optional transform as input.
    - It reiterates the process of collecting and intersecting sample IDs to ensure only complete samples are included.
- **__len__**: Returns the total number of complete samples.

- **__getitem__**:
    - Given an index, it retrieves the image path (checking for .jpg then .png).
    - Loads the image using PIL.Image and converts it to RGB.
    - Applies the specified transform to the image.
    - Loads the corresponding _exp.npy, _val.npy, _aro.npy, and _lnd.npy files.
    - Returns the processed image, expression (as target_class), [valence, arousal] (as target_reg), and landmarks.

4. Data Transformations: torchvision.transforms.Compose is used to define a series of transformations applied to the images:

- Resize((224, 224)): Resizes images to a standard input size for many pre-trained models.
- RandomHorizontalFlip(): Augments data by randomly flipping images horizontally.
- ToTensor(): Converts PIL images to PyTorch tensors.
- Normalize(...): Normalizes image pixel values using standard mean and standard deviation for ImageNet pre-trained models.

5. DataLoader Setup:

- The AffectDataset is instantiated with the image and annotation directories and the defined transformations.
- The dataset is split into training and validation sets using random_split based on the VAL_SPLIT ratio from CONFIG.
- DataLoader instances are created for both training and validation sets, handling batching, shuffling (for training), and parallel data loading (num_workers).

Models (ResNet + VGG)

This section defines the deep learning models used for multi-task learning.

1. MultiTaskResNet:

- Uses resnet50 pre-trained on ImageNet (weights="IMAGENET1K_V1") as its base encoder.
- The original fully connected layer (base.fc) is replaced with nn.Identity() to extract features.

- Two new linear layers are added:
    - classifier: For expression classification (num_classes outputs).
    - regressor: For valence/arousal regression (2 outputs).
- The forward method passes the input through the encoder and then through both the classifier and regressor heads.

2. MultiTaskVGG16:

- Uses vgg16 pre-trained on ImageNet as its base encoder.
- The original classifier block is replaced with nn.Identity() to extract features.
- Two new sequential blocks are added for:
    - classifier: A small MLP with ReLU and Dropout layers, ending in num_classes outputs.
    - regressor: Another small MLP with ReLU and Dropout layers, ending in 2 outputs.
- The forward method flattens the encoder features if necessary and then passes them through both the classifier and regressor heads.

Training Loop

This section defines the core training and validation process.

1. train_model function:

- Takes the model, train_loader, val_loader, optimizer, criterion_class (e.g., CrossEntropyLoss), criterion_reg (e.g., MSELoss), device, and epochs as input.
- Initializes a history dictionary to record training and validation losses.
- Iterates through the specified number of epochs.
- **Training Phase (model.train()):**
    - Iterates through batches from train_loader.
    - Moves images, expression labels, and regression targets to the specified device.
    - Performs a forward pass to get classification and regression outputs.
    - Calculates loss_c (classification loss) and loss_r (regression loss).

    - Combines them into a total loss.
    - Performs backpropagation (loss.backward()) and updates model weights (optimizer.step()).
    - Accumulates train_loss.
- **Validation Phase (model.eval()):**
    - Disables gradient calculation (torch.no_grad()).
    - Iterates through batches from val_loader, performs a forward pass, and accumulates val_loss.
- Prints epoch-wise average training and validation losses.
- Returns the history of losses.

2. plot_history function:

- Takes the history dictionary and a title as input.
- Uses matplotlib.pyplot to plot the training and validation loss curves, providing a visual overview of model performance during training.

Metrics

This section defines various evaluation metrics for both classification and regression tasks.

1. Regression Metrics:

- rmse: Root Mean Squared Error.
- corr: Pearson correlation coefficient.
- sagr: Sign Agreement Rate.
- ccc: Concordance Correlation Coefficient.

2. Classification Metrics:

- krippendorffs_alpha: Krippendorff's Alpha, a measure of inter-rater reliability.
- classification_metrics: A comprehensive function that calculates:
    - Accuracy
    - F1-Score (weighted)
    - Cohen's Kappa

- AUC (Area Under ROC Curve, macro-averaged, OvR)
- AUC-PR (Area Under Precision-Recall Curve)

3. evaluate_model function:

- Takes the model, loader, and device as input.
- Sets the model to evaluation mode (model.eval()) and disables gradient calculation.
- Iterates through batches from the provided loader.
- Collects true and predicted values for both classification (y_true_cls, y_pred_cls, y_prob_cls) and regression (y_true_val, y_pred_val, y_true_aro, y_pred_aro).
- Calculates and returns a dictionary of classification metrics and another dictionary of regression metrics.

Visualization

This section provides a utility for visualizing model predictions.

1. show_predictions function:

- Takes the model, dataset, device, and class_names (e.g., a list of emotion labels) as input.
- Creates a grid of subplots (2x3).
- For each subplot, it randomly selects a sample from the dataset.
- Passes the image through the model to get classification probabilities and regression predictions.
- Displays the image with its ground truth class, predicted class, and predicted valence/arousal values in the title.

Main Run

This section orchestrates the entire training and evaluation process.

1. Device Setup: Determines the computing device (CUDA if available, else CPU).

2. Data Loading & Transforms: Re-initializes AffectDataset with image/annotation directories and a slightly enhanced transform including ColorJitter.

3. Subset Usage: Includes an important line to create a torch.utils.data.Subset of the dataset (subset_size = 1500). This is crucial for faster iteration and development in Colab, especially with limited GPU resources. **Note:** For full training, this line should be commented out or subset_size increased.

4. Train/Validation Split & DataLoaders: Splits the (possibly subsetted) dataset and creates DataLoader instances for training and validation.

5. Loss Functions: Defines nn.CrossEntropyLoss() for classification and nn.MSELoss() for regression.

6. Model Training and Evaluation Loop:

- Iterates through the two defined models: "ResNet50" (MultiTaskResNet) and "VGG16" (MultiTaskVGG16).
- For each model:
    - Instantiates the model and moves it to the device.
    - Initializes an Adam optimizer.
    - Calls train_model to train the model for a specified number of epochs (e.g., 5).
    - Calls plot_history to visualize the training curves.
    - Calls evaluate_model on the validation set to get classification and regression metrics.
    - Stores the results in a results dictionary.

7. Results Comparison:

- After training both models, it uses pandas.DataFrame to neatly display and compare the classification and regression metrics for ResNet50 and VGG16, rounding values for cleaner output.